Bridge

dge · Draw shapes with Vector vs Raster renderers independently · Build with Wa

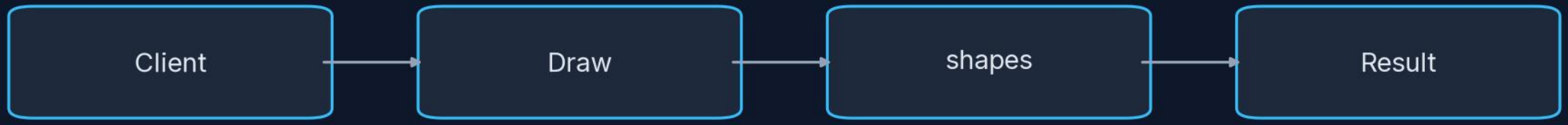
Today's Hook

- Problem: Draw shapes with Vector vs Raster renderers independently
- Pattern: Bridge — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Draw shapes with Vector vs Raster renderers independently
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Bridge: Draw shapes with Vector vs Raster renderers indepe

Real-World Scenario

- Running example: Draw shapes with Vector vs Raster renderers independently
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.gof.structural.bridge.problem;

import java.util.*;

public final class NaiveShapes {
  public static String vectorCircle(int r){ return "vector-circle r="+r; }
  public static String rasterCircle(int r){ return "raster-circle r="+r; }
  public static String vectorSquare(int s){ return "vector-square s="+s; }
  // explosion of combinations
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.gof.structural.bridge.problem;

import java.util.*;

interface Renderer { String draw(String shape); }
record VectorRenderer() implements Renderer { public String draw(String s){ return "vector-"+s; } }
record RasterRenderer() implements Renderer { public String draw(String s){ return "raster-"+s; } }
abstract class Shape {
  protected final Renderer r; protected Shape(Renderer r){ this.r=r; }
  public abstract String draw();
}
final class Circle extends Shape {
  private final int radius; public Circle(Renderer r, int radius){ super(r); this.radius=radius; }
  public String draw(){ return r.draw("circle r="+radius); }
}
final class Square extends Shape {
  private final int side; public Square(Renderer r, int side){ super(r); this.side=side; }
  public String draw(){ return r.draw("square s="+side); }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.gof.structural.bridge.problem;

import java.util.*;

public final class ProblemDemo {
  public static void main(String[] args) {
    System.out.println("=== Bridge: shape × renderer ===");
    System.out.println("--- naive ---");
System.out.println(NaiveShapes.vectorCircle(3));
System.out.println("--- bridge ---");
System.out.println(new Circle(new VectorRenderer(), 3).draw());
System.out.println(new Square(new RasterRenderer(), 4).draw());
  }
}
```

Walkthrough Output

```
=== Bridge: shape × renderer ===
--- naive ---
vector-circle r=3
--- bridge ---
vector-circle r=3
raster-square s=4
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Bridge.

Related Patterns

- Contrast Bridge with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Bridge → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.